



République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Département IA&SD

Spécialité : SII

Méta-heuristiques et Algorithmes Evolutionnaires

Problème du Sacs à Dos Multiples

Rapport de Projet

Présenté par : Merabti Amar

Présenté à : Naila Houacine

Le 16/05/2024

Master 1 SII 2023/24

1. Introduction

Les métaheuristiques sont des outils essentiels dans la résolution de problèmes d'optimisation, offrant des solutions approximatives dans des délais raisonnables, particulièrement lorsque les approches traditionnelles atteignent leurs limites en raison de la taille ou de la complexité du problème. Dans cette deuxième partie du projet, nous nous concentrons sur le problème des Sacs à Dos Multiples (Multiple Knapsack Problem). Notre objectif est d'explorer deux métaheuristiques, à savoir l'algorithme génétique (GA : Genetic Algorithm) et l'optimisation par essaim de particules (BSO : Bee Swarm Optimization), pour résoudre ce problème complexe.

Dans ce rapport, nous commencerons par une description générale sur chaque métaheuristique. Ensuite, nous expliquerons le fonctionnement de chaque métaheuristique, en mettant en évidence leur inspiration et les algorithmes associés. Nous décrirons également nos expériences concernant le paramétrage de chaque métaheuristique, ainsi que l'impact de différentes tailles de problème sur leurs performances. Enfin, nous comparerons les résultats obtenus avec les métaheuristiques entre elles, ainsi qu'avec les résultats de la première partie du projet. Nous conclurons par une synthèse des principales observations et des perspectives futures.

2. Algorithme évolutif : Algorithme Génétique

a. Description

Les algorithmes évolutifs sont inspirés du processus évolutif naturel, où les solutions candidates sont traitées comme des individus dans une population. L'algorithme génétique (GA) est l'une des techniques les plus courantes dans cette catégorie. Il repose sur les principes de sélection naturelle, de recombinaison génétique et de mutation pour évoluer vers des solutions optimales.

b. Fonctionnement

Initialisation de la population :

Une population initiale de solutions candidates est générée aléatoirement. Dans le cas du Sac à Dos Multiple, on essaie d'affecter à chaque objet un sac d'une façon aléatoire, tout en vérifiant qu'on peut ajouter cet objet à ce sac, sinon on va ignorer cet objet et il ne va pas être placé.

Évaluation de la population :

Chaque individu de la population est évalué en fonction de sa capacité à résoudre le problème, mesurée par une fonction d'évaluation. Une fonction d'évaluation pour le problème du MPK consiste à calculer la valeur totale des objets qui sont placés dans des sacs, en faisant la somme des valeurs de chacun de ces objets.

Sélection des parents :

Les individus les mieux adaptés sont sélectionnés pour la reproduction, basée généralement sur des méthodes telles que la roulette, le tournoi ou la sélection élitiste. Dans notre implémentation on a utilisé la sélection élitiste, où on sélectionne à chaque itération un certain nombre de solutions élites (facteur de sélection -voir la section paramétrage-), ces solutions sélectionnées seront passées pour à l'étape suivante qui est le croisement.

Croisement (Crossover) :

Le croisement est l'une des étapes cruciales de l'algorithme génétique où les informations génétiques sont échangées entre les parents sélectionnés pour créer une descendance. Il existe plusieurs techniques de croisement, chacune ayant ses propres avantages et inconvénients, dans notre implémentation on a utilisé le Croisement en un point (Single-point crossover), où un point de croisement est choisi aléatoirement le long des chromosomes parentaux, et les informations génétiques sont échangées à partir de ce point pour créer des enfants.

Mutation :

La mutation est un autre opérateur génétique utilisé dans l'algorithme génétique pour introduire une diversité génétique dans la population, permettant ainsi une exploration plus complète de l'espace des solutions. Voici comment la mutation fonctionne généralement : on définit une probabilité de mutation (seuil), puis pour chaque solution on génère un nombre aléatoire (entre 0 et 1) si ce nombre dépasse le seuil on la mutation sinon on passe à la solution suivante. Dans le cas où la mutation est appliqué, on choisit un objet aléatoire de notre solution, lequel on affecte un nouveau sac dont il va être placé.

Remplacement :

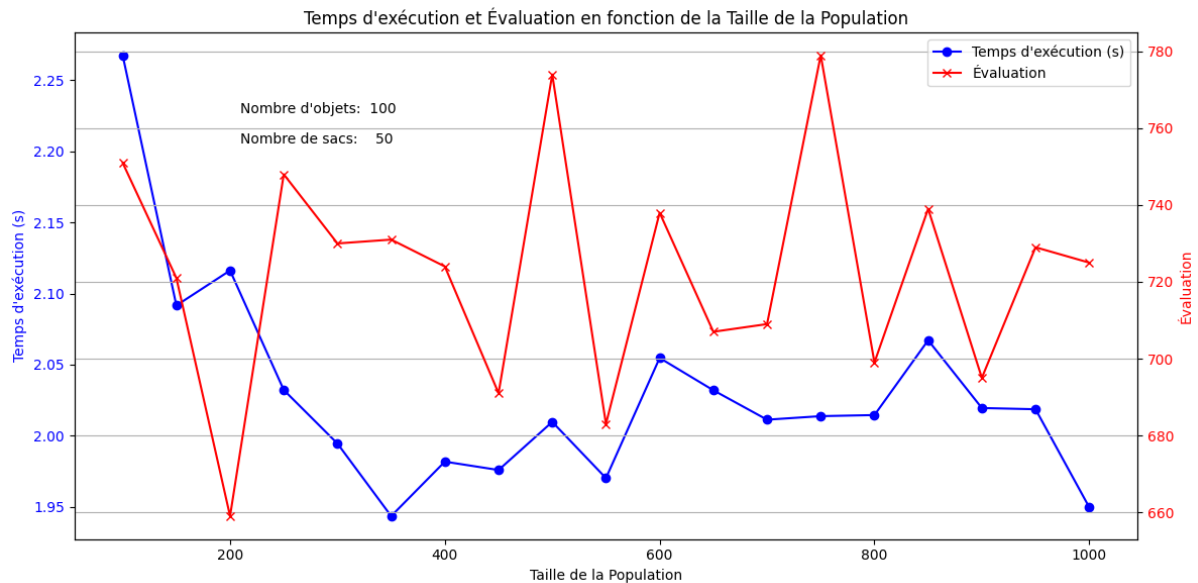
Après l'application des opérateurs génétiques précédents, les nouveaux individus remplacent ceux de la génération précédente, en suivant des stratégies telles que le remplacement des pires ou le remplacement aléatoire. Dans notre implémentation on a remplacé les pires solutions par les nouvelles solutions.

c. Paramétrage

Taille de la population :

La taille de la population détermine le nombre d'individus dans chaque génération. Une taille de population plus grande permet généralement à l'algorithme de mieux explorer l'espace des solutions, mais cela nécessite également plus de ressources informatiques. Le choix de la taille de la population dépend de la complexité du problème, de la nature de l'espace de recherche et des contraintes de temps et de mémoire.

L'expérimentation sur ce paramètre nous a permis d'obtenir les résultats suivants :



Analyse des résultats :

Globalement, le temps d'exécution semble rester relativement stable à mesure que la taille de la population varie. Cela suggère que le temps nécessaire pour exécuter l'algorithme génétique n'est pas fortement influencé par la taille de la population dans ce cas spécifique.

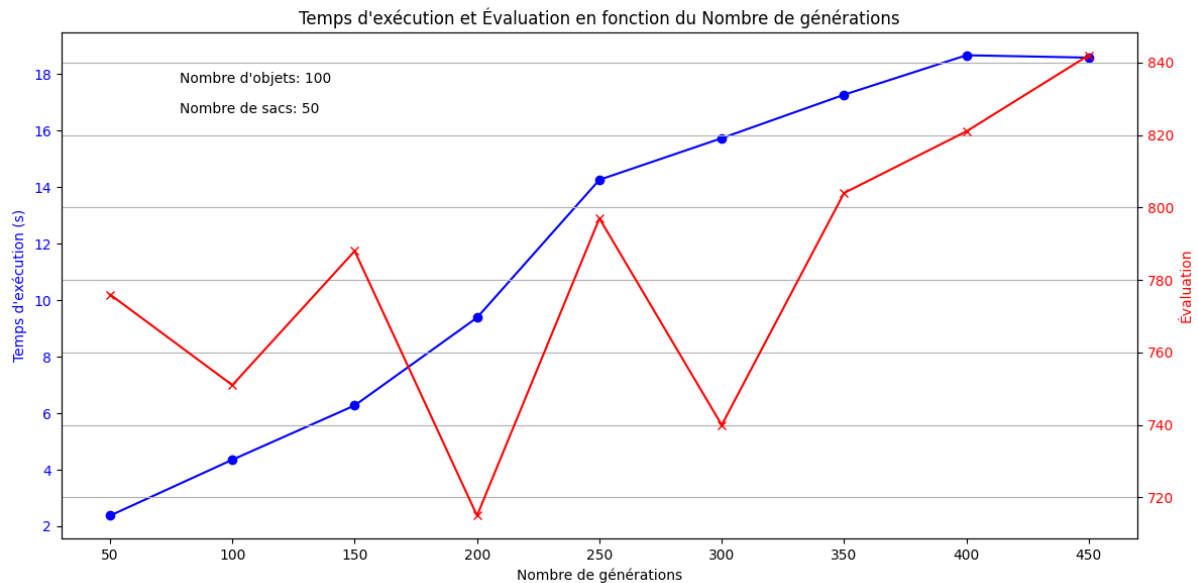
L'évaluation de la solution fluctue légèrement en fonction de la taille de la population. Il n'y a pas de tendance claire montrant une amélioration ou une détérioration constante de la qualité de la solution à mesure que la taille de la population change. Cependant, il y a des variations dans les performances de l'algorithme pour différentes tailles de population.

On observe une certaine stabilité dans les performances de l'algorithme génétique malgré les variations de la taille de la population. Cela suggère que, dans ce cas, la taille de la population seule n'est peut-être pas un facteur critique pour obtenir des solutions de haute qualité dans un délai raisonnable.

Nombre de générations :

Le nombre de générations détermine le nombre d'itérations que l'algorithme génétique effectuera avant de s'arrêter. Un nombre de générations plus élevé permet généralement à l'algorithme de mieux explorer l'espace des solutions, mais cela peut également entraîner un temps de calcul plus long. Ce paramètre doit être choisi en fonction de la complexité du problème et des ressources informatiques disponibles.

Pour cela on a fait une expérimentation sur ce paramètre, où en utilisant la même instance du problème on varie le nombre d'itération à faire. Les résultats sont résumés dans le graphe suivant :



Analyse des résultats :

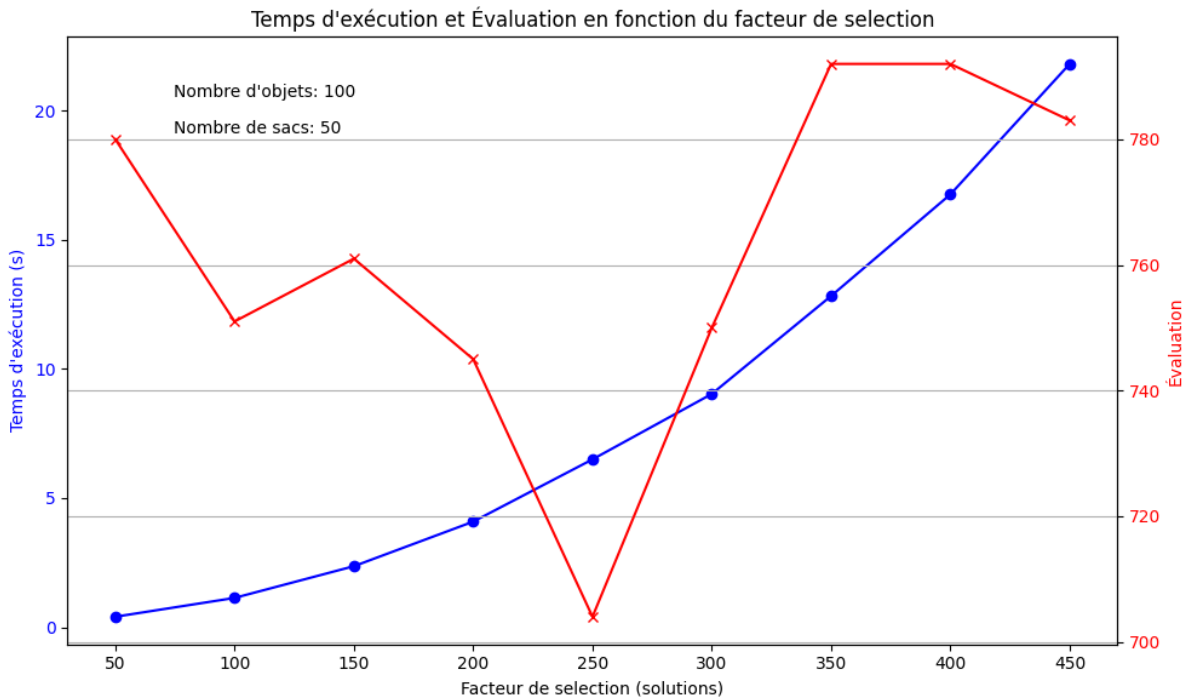
Le nombre de générations varie de 50 à 450. On peut remarquer que le temps d'exécution augmenter de manière linéaire avec le nombre de générations. Cela peut indiquer une complexité temporelle croissante de l'algorithme génétique à mesure qu'il recherche des solutions sur un plus grand nombre de générations.

Cependant, il n'y a pas de tendance claire en ce qui concerne l'évaluation de la solution en fonction du nombre de générations. Cela suggère que le nombre de générations seul ne détermine pas directement la qualité de la solution trouvée par l'algorithme génétique. Mais ça tendance générale montre que la qualité d'une solution peut être améliorer si le nombre de génération augmente.

Facteur de sélection :

Le facteur de sélection détermine la méthode utilisée pour sélectionner les parents pour la reproduction. Des méthodes couramment utilisées incluent la sélection par roulette, la sélection élitiste et la sélection par tournoi. Le choix du facteur de sélection peut avoir un impact significatif sur la convergence de l'algorithme et sur sa capacité à éviter les optimaux locaux.

Dans notre implémentation la sélection élitiste est utilisée, en fixant la taille de la population à 1000 et en faisant varier le nombre de solutions élitistes à sélectionner on a pu obtenir ces résultats :



Analyse des résultats :

Le temps d'exécution augmente de manière significative à mesure que le facteur de sélection augmente. Cela suggère que la sélection d'un plus grand nombre de solutions élites entraîne une augmentation du temps nécessaire pour exécuter l'algorithme génétique.

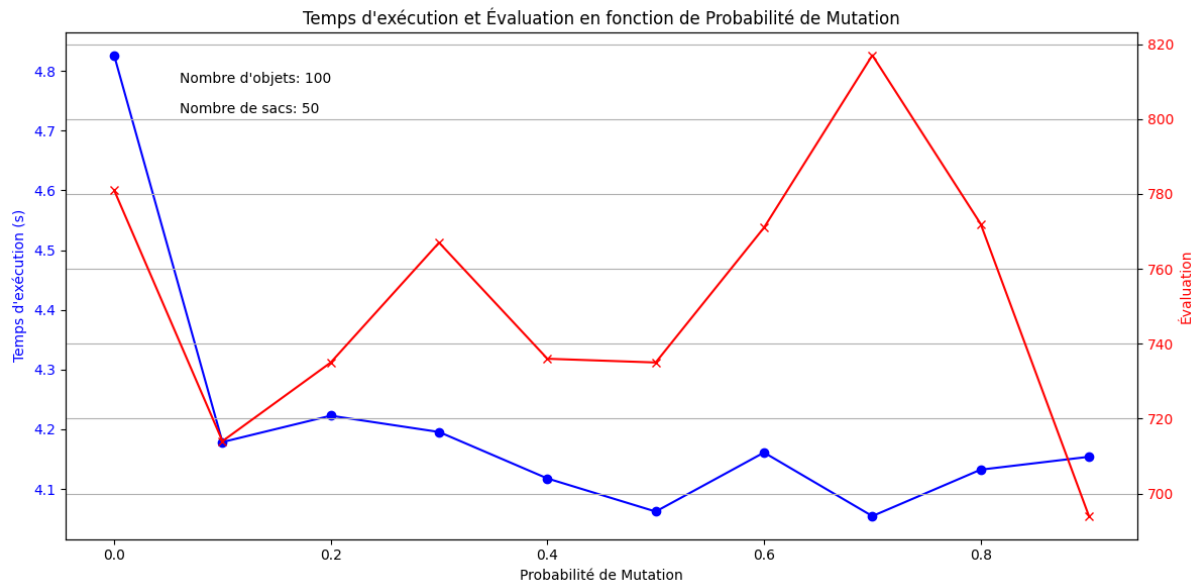
L'augmentation du facteur de sélection a une légère influence sur la qualité de la solution trouvée, où cette dernière peut être améliorée en choisissant un facteur de sélection plus grand.

Le choix du facteur de sélection est crucial pour équilibrer l'exploration de l'espace des solutions et l'exploitation des solutions existantes. Un facteur de sélection moins élevé favorise généralement l'exploitation en privilégiant les solutions de haute qualité, mais cela peut également réduire l'exploration de l'espace des solutions et conduire à une convergence prématurée vers des solutions sous-optimales.

Probabilité de mutation :

La probabilité de mutation contrôle la fréquence à laquelle les gènes sont modifiés dans la population. Une probabilité de mutation plus élevée favorise une exploration plus diversifiée de l'espace des solutions, tandis qu'une probabilité plus faible peut conduire à une convergence prématurée vers un optimum local. Il est important de choisir une probabilité de mutation appropriée qui permet à l'algorithme de maintenir un équilibre entre l'exploration et l'exploitation.

Pour cela une expérimentation des différentes valeurs de cette probabilité est nécessaire, voici notre résultat obtenu :



Analyse des résultats :

Le temps d'exécution semble relativement stable à mesure que la probabilité de mutation varie. Il n'y a pas de tendance claire montrant une augmentation ou une diminution significative du temps d'exécution en fonction de la probabilité de mutation.

Cependant pour la qualité de la solution, on peut observer clairement que les valeurs de la probabilité qui $\in [0.6, 0.8]$ donnent des solutions plus optimales que d'autres valeurs, ce qui fait que l'équilibre de l'exploitation et l'exploration est assuré au maximum dans cet intervalle.

3. Algorithme d'intelligence en essaim : BSO

a. Description

L'optimisation par essaim d'abeille (Bee Swarm Optimization, BSO) est une métaheuristique inspirée du comportement collectif des colonies d'abeilles lors de la recherche de nourriture. Les abeilles communiquent entre elles pour explorer et exploiter efficacement les sources de nourriture, ce qui permet de trouver des solutions optimales de manière collective. En appliquant ces principes, l'algorithme BSO utilise une population d'agents (les abeilles) pour explorer l'espace des solutions d'un problème d'optimisation.

b. Fonctionnement

Initialisation :

L'algorithme BSO commence par initialiser une solution de référence appelé Sref, qui représente la meilleure solution actuelle. Cette initialisation se fait d'une manière aléatoire.

Détermination des points de recherche :

Après la phase d'initialisation, la solution de référence est ajoutée à une liste taboue, puis, à partir de cette Sref, on génère des points de recherches. La détermination des points de recherche se fait de la manière suivante :

On applique un flip sur la Sref, où ce flip est un paramètre empirique. Le flip consiste à inverser des éléments de la solution actuelle qui sont séparés par flip éléments, par exemple dans le cas du problème du sac à dos multiple, un flip = 5 consiste à changer le sac de chaque 5 objets de la solution par un autre sac aléatoire (c'est par exemple la taille de solution est 15, le flip est appliquer 3 fois).

Cette opération de flip est appliquée un certain nombre de fois, ce nombre est considéré comme le nombre d'abeille utilisé dans notre algorithme. Donc à chaque fois on applique le flip on génère un point de recherche qui sera assigné à une abeille.

Recherche locale :

Une fois tous les points de recherche seront assignés aux abeilles, ces dernières commencent leurs recherches en locale.

La recherche locale consiste à explorer le voisinage de chaque solution afin de déterminer des zones potentielles prometteuses. Donc chaque abeille essaie de trouver une solution meilleure que celle qui est assignée au début. Cette recherche est faite un certain nombre de fois qu'on appelle le nombre d'itération locales.

Une fois l'abeille termine sa recherche, elle insère dans une table Danse sa meilleure solution trouvée. La table Danse alors va contenir les meilleures solutions trouvées par chaque abeille.

Mise à jour de Sref :

Une fois la recherche locale est terminée et la table Danse sera remplie, on fait une mise à jour à notre solution de référence en suivant les étapes suivantes :

On calcule une valeur $\Delta f = f(\text{bestSol}(\text{Danse})) - f(\text{Sref})$ d'où on a deux cas :

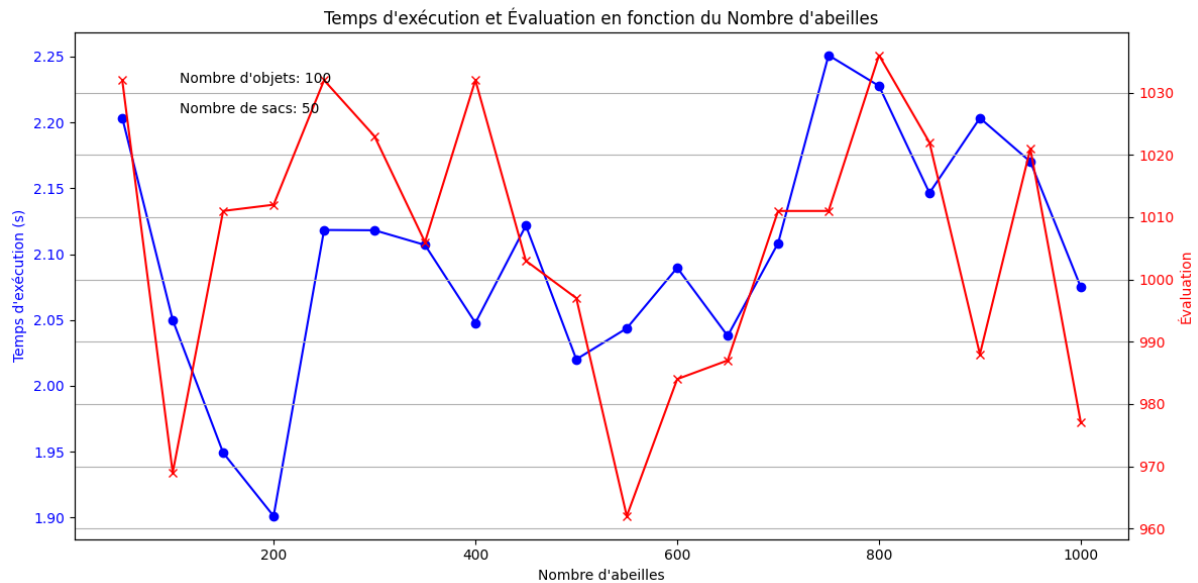
- cas $\Delta f > 0$: on affecte la meilleure solution en terme de qualité à Sref
- cas sinon : on diminue le nombre de chances de Sref par 1 et on vérifie si elle a consommé toutes ces chances, si c'est le cas l'algorithme préfère de viser vers l'exploration en affectant la solution la plus diversifiée à Sref, pour ne pas rester dans les optimum locaux et réassigne le nombre de chances au nombre de chances maximale (paramètre empirique), sinon (la solution elle a encore des chances), la meilleure solution en terme de qualité sera affecter à Sref.

c. Paramétrage

Nombre d'abeilles :

La taille de la population détermine le nombre d'abeilles dans l'essaim. Une taille de population plus grande permet une exploration plus étendue de l'espace des solutions, mais augmente également le coût computationnel. Le choix de ce paramètre dépend de la complexité du problème et des ressources disponibles.

Expérimentation sur ce paramètre :



Analyse de résultats :

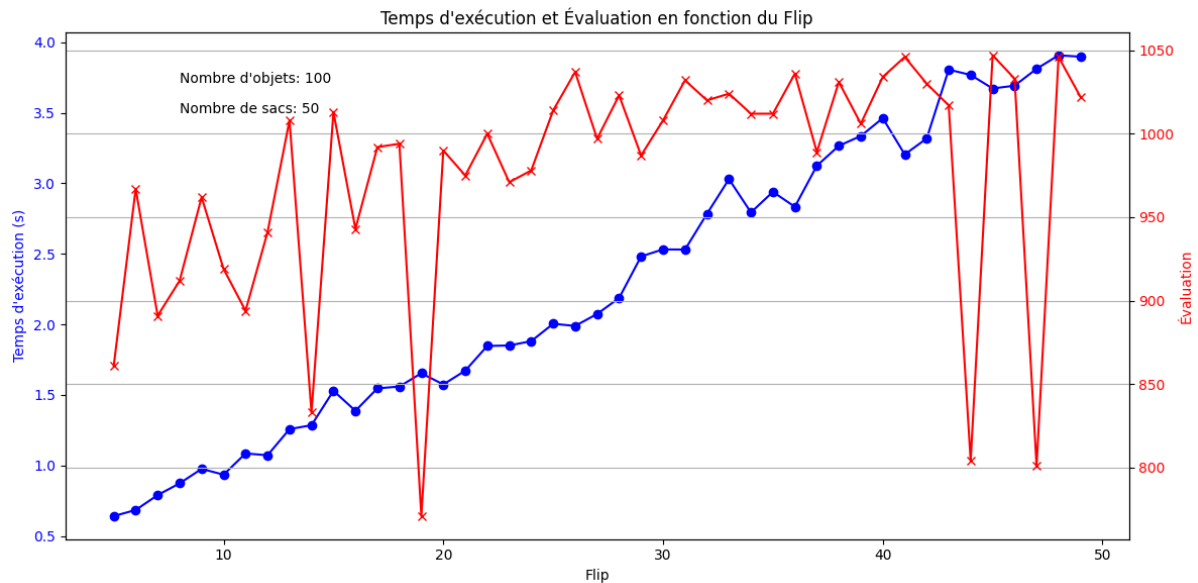
Le temps d'exécution reste relativement stable autour de 2 secondes pour la plupart des valeurs du nombre d'abeilles. Il y a de légères variations, mais aucune tendance claire indiquant une augmentation ou une diminution systématique du temps d'exécution en fonction du nombre d'abeilles.

La stabilité des performances en termes de temps d'exécution et d'évaluation de la solution suggère que l'algorithme est robuste par rapport aux variations du nombre d'abeilles. Il semble capable de maintenir des performances élevées sur une large gamme de valeurs.

Il n'y a pas de point optimal évident pour le nombre d'abeilles, mais des nombres entre 50 et 400 semblent offrir un bon compromis entre temps d'exécution et qualité de la solution. Des valeurs plus élevées ne montrent pas de gain significatif en termes de qualité de solution.

Flip :

Le paramètre flip est essentiel pour le fonctionnement de BSO, il contrôle la génération des points de recherche et la façon dont la solution de référence est exploitée pour générer des Search Area mieux adaptés au problème. Voici une exécution de BSO avec des différentes valeur de flip :



Analyse des résultats :

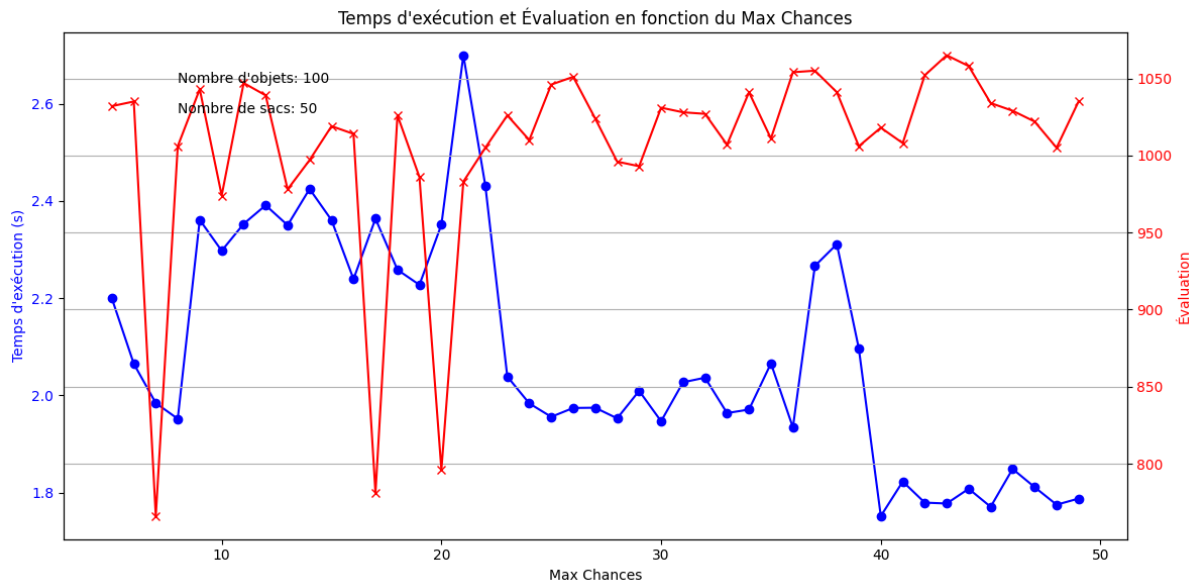
Le temps d'exécution augmente de manière assez constante avec l'augmentation de la valeur de "flip". Cela est logique car plus d'éléments à modifier nécessite plus de calculs et de vérifications, augmentant ainsi le temps d'exécution.

L'augmentation du flip semble généralement améliorer les évaluations de la solution jusqu'à un certain point, après quoi les gains se stabilisent ou même diminuent avec des oscillations importantes. Par exemple, les évaluations augmentent globalement jusqu'à un flip de 45, mais il y a des fluctuations importantes pour les valeurs plus élevées.

Une plage de 20 à 40 flips semble offrir un bon compromis entre temps d'exécution et qualité de la solution, avec des évaluations élevées et des temps d'exécution gérables. Les évaluations les plus élevées sont obtenues pour des flips autour de 45, mais avec des risques d'instabilité pour des flips encore plus élevés.

Max Chances :

Ce paramètre indique le nombre de chance qu'une solution peut avoir avant d'être remplacée par une autre, c'est un paramètre empirique sensible donc il faut le choisir soigneusement. Pour celle on a fait une expérimentation sur les différentes valeurs de ce paramètre :



Analyse des résultats :

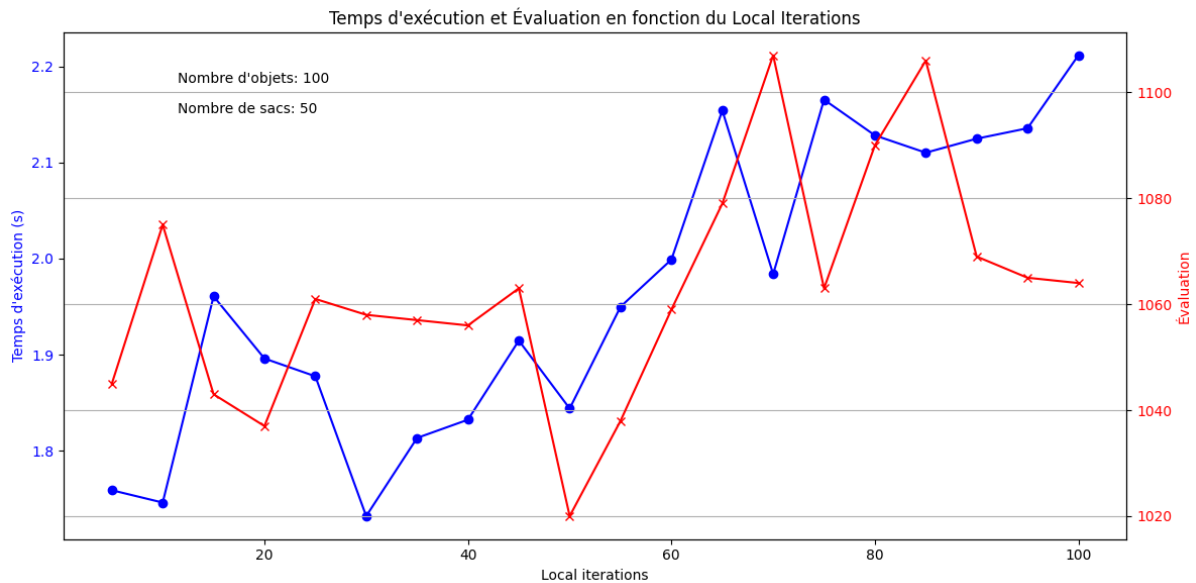
Le temps d'exécution varie légèrement en fonction du nombre de tentatives, mais il reste globalement stable autour de 2 secondes, avec quelques variations mineures. Cela suggère que le nombre maximum de tentatives n'a pas un impact significatif sur le temps d'exécution global de l'algorithme.

Les plus faibles évaluations se produisent avec un nombre relativement faible de tentatives et avec des oscillations remarquables, ce qui peut suggérer qu'un nombre insuffisant de tentatives limite la capacité de l'algorithme à exploiter suffisamment l'espace des solutions.

Bien que le nombre optimal de tentatives puisse dépendre des spécificités du problème, il semble qu'une plage de 30 à 50 tentatives offre un bon compromis entre temps d'exécution et qualité de la solution, en se basant sur les évaluations élevées observées dans cette plage.

Nombre d'itérations locales :

C'est le nombre d'itération que chaque abeille va faire dans son propre point de recherche, ce paramètre peut considérablement influencer les temps d'exécutions et la qualité des solutions c'est il n'est pas bien choisi. Les résultats de l'expérimentation suivante peuvent nous donner une idée sur cette influence :



Analyse des résultats :

Le temps d'exécution varie en fonction du nombre d'itérations locales, augmentant globalement avec le nombre d'itérations locales. Cela suggère que plus d'itérations locales augmentent le temps d'exécution, ce qui est attendu puisque davantage de calculs sont effectués.

Les évaluations sont relativement stables pour des nombres d'itérations locales compris entre 25 et 100, oscillant autour de 1060 à 1100. Cela montre que l'algorithme est robuste et peut maintenir de bonnes performances sur une large plage de valeurs d'itérations locales.

Les résultats montrent qu'une plage de 65 à 85 itérations locales offre un bon compromis entre temps d'exécution et qualité de la solution, avec des évaluations atteignant les valeurs les plus élevées dans cette plage.

4. Expérimentation : GA vs BSO

Dans cette partie, on va procéder à faire une comparaison entre l'algorithme génétique et l'algorithme BSO, en utilisant les mêmes instances du problème en fixant en premier le nombre d'objets à 100 et de varier le nombre de sacs de 20 à 80 sacs. Puis une autre exécution où on fixe cette fois ci le nombre de sacs à 20 et de varier le nombre d'objets de 20 à 100.

Voici les paramètres des deux algorithmes utilisés pour l'expérimentation :

GA :

Taille de la population : 500

Nombre de génération max : 100

Facteur de sélection : 200

Probabilité de mutation : 0.7

BSO :

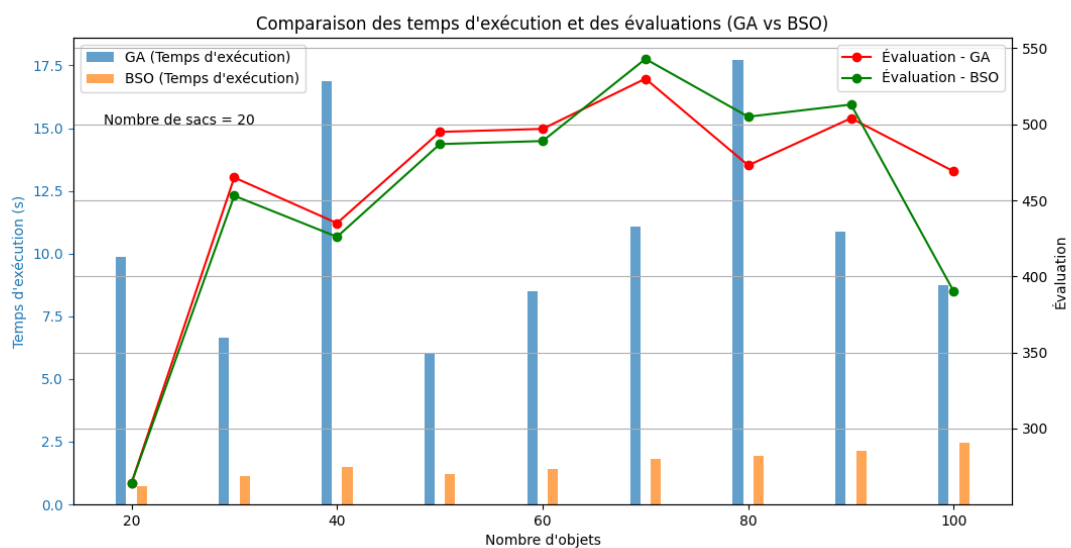
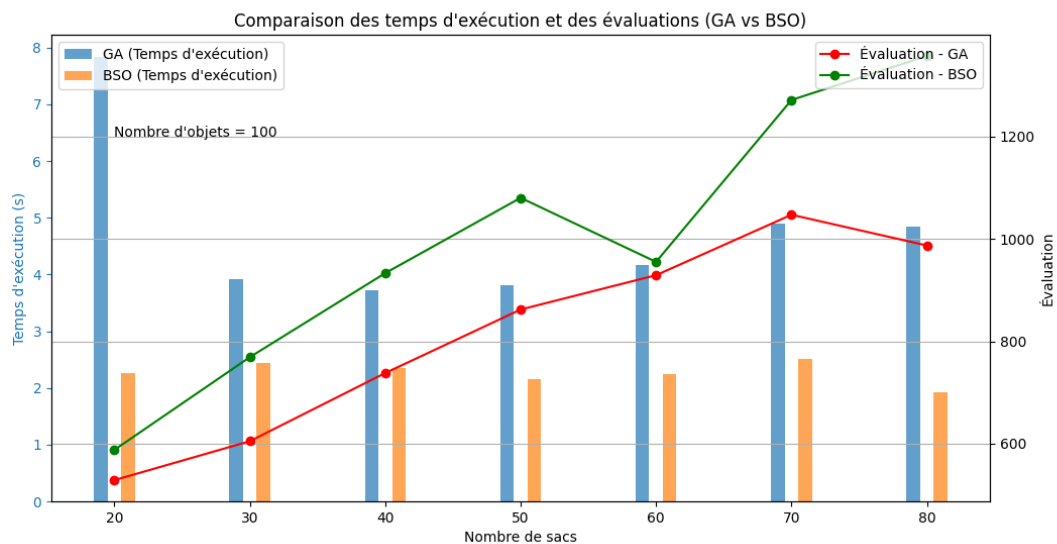
Nombre d'itération : 1000

Flip : 25

Max Chances : 30

Nombre d'abeilles : 1000

Nombre d'itérations locales : 65



Analyse des résultats :

GA a un temps d'exécution significativement plus long que BSO pour toutes les instances. Par exemple, pour 20 objets et 20 sacs, le temps d'exécution est de 9.8815 secondes pour BSO contre seulement 0.7273 secondes pour BSO. Cette tendance se maintient pour toutes les autres instances également.

BSO est beaucoup plus rapide, avec des temps d'exécution généralement inférieurs à 3 secondes, même pour les instances les plus grandes (100 objets et 80 sacs), où GA prend entre 3.7 et 17.7 secondes.

Les évaluations des solutions produites par les deux algorithmes varient, avec une tendance générale indiquant que BSO produit des évaluations généralement meilleures ou comparables à celles de GA.

Dans la première exécution, BSO non seulement obtient de meilleures évaluations mais le fait en un temps beaucoup plus court. Par exemple, pour 100 objets et 80 sacs, BSO a une évaluation de 1357 contre 987 pour GA.

Récapitulatif :

BSO est nettement plus efficace en termes de temps d'exécution pour toutes les instances testées, ce qui le rend plus adapté pour des problèmes où le temps de calcul est crucial.

En termes de qualité des solutions, BSO fournit généralement de meilleures évaluations, particulièrement pour les instances plus grandes, indiquant qu'il est non seulement plus rapide mais aussi potentiellement plus efficace en termes de qualité de la solution trouvée.

BSO, bien que produisant des solutions de qualité raisonnable, est considérablement plus lent, ce qui peut le rendre moins pratique pour des problèmes de grande taille ou lorsque le temps de réponse est un facteur critique.

5. Comparaison avec les algorithmes de la partie 1

En comparant ces algorithmes aux celles de la partie 1, on voit clairement que les algorithmes évolutif et d'intelligence en essaim sont incomparablement plus efficaces que les méthodes basées espace de recherche (méthodes aveugles : DFS, BFS et heuristiques : A*), notamment sur les tailles du problème assez grandes, pour l'instance avec les méthodes de la 1^{er} partie on n'a pu même pas exécuter sur des tailles qui dépassent les 10 objets et les 10 sacs dans un temps raisonnable, mais avec ces algorithmes évolutif et Swarm intelligence on peut facilement résoudre des problèmes de grande taille en des fractions de secondes, c'est la puissance des algorithmes basés espaces de solutions.

6. Conclusion

En conclusion, durant ce travail on a pu explorer plusieurs approches pour résoudre le problème du Sac à Dos Multiple.

Dans la première partie de ce projet on a vu les approches basées espace de recherche avec les méthodes aveugle DFS et BFS, mais aussi un algorithme heuristique A*, où on a découvert les limites de ces approches et la nécessité de s'orienter vers les métaheuristiques quand la taille du problème à résoudre serait exponentiellement croissante.

La deuxième partie a été consacrer dans l'étude des approche basé espace de solution, dans laquelle on a vu les algorithmes évolutifs tel que l'algorithme génétique, et les algorithmes d'intelligence en essaim où on a vu l'algorithme d'optimisation en essaim d'abeille. Cette partie nous a permet de découvrir la puissance des métaheuristiques quand la taille du problème a résoudre est de nature exponentielle, ces algorithmes nous a permet de résoudre le problème du Sac à Dos Multiple a des tailles qui peut dépasser les 100 objets et 100 sacs facilement, en jouant sur les différent paramètres empiriques de chaque algorithmes, et de voir et comparer les résultat obtenus.

Enfin, cette expérimentation a montré que l'algorithme BSO est plus efficace que l'algorithme génétique, soit en termes de qualité des solutions soit en termes des temps d'exécutions. Cependant une étude plus approfondie sur les paramètres de chaque algorithme est nécessaire pour bien exploiter toute la puissance que ces des deux peuvent offrir.